Authentication, Sessions and Test Isolation

LESSON SPECIFICATIONS & ROADMAP

• Track: Playwright Automation with JavaScript + AI • Level: Intermediate to Advanced
• Estimated Effort: 2.5–3 Hours • Prerequisites: Days 1–10 concepts (Locators, web-first assertions, auto-waiting, fixtures,
• Runtime: Pure Modern JavaScript (.js) • Runner: @playwright/test

Master production authentication testing in Playwright. Bypass repetitive login UI clicks using storageState session caching, enforce strict test isolation, and protect credentials with environment variables.

Day 11 Learning Objectives

'Understand authentication mechanisms: sessions, cookies, JWTs, localStorage, and role-based access control (RBAC).
'Test core authentication workflows: successful login, invalid credentials, logout, and session persistence.
'Test authorization boundaries: verifying redirects or 403 Forbidden when unauthenticated users access protected routes.
'Implement Playwright storageState to log in once and reuse authenticated browser state across hundreds of tests.
'Configure authentication setup projects in playwright.config.js.
'Maintain strict test isolation and avoid shared mutable state anti-patterns across concurrent worker threads.
'Manage secrets securely using environment variables and ensure authentication state files are gitignored.

Modern web applications protect sensitive data and features behind authentication and authorization layers.

Authentication vs Authorization
- **Authentication (AuthN): Who are you? (e.g. Logging in with username and password, verifying MFA tokens).**
- **Authorization (AuthZ): What are you allowed to do? (e.g. Can a Student delete a course? Can an Admin approve a refund?).**

How Browsers Store Session State
After a successful login, web servers establish an authenticated session using:
1. **HTTP-only Cookies: Secure tokens stored by the browser that are automatically attached to outgoing HTTP requests.**
2. **Local Storage / Session Storage: Client-side storage storing JWT (JSON Web Tokens) or session IDs.**

Role-Based Access Control (RBAC)
Enterprises define roles such as:
- **Admin: Full access to user management, billing, and system configuration.**
- **Instructor / Manager: Access to content creation and student grading.**
- **Student / Customer: Read-only access to enrolled content and personal profile.**
- **Guest (Unauthenticated): Access restricted to landing page and login portal.**

Protected Route Redirection Logic (A typical web app redirecting unauthenticated visitors.)

```
// If an unauthenticated user navigates directly to /settings
// Expect redirect: /login?redirectTo=/settings
```

Every robust test suite must verify the core authentication lifecycle:

1. Successful Login Happy Path
Verify that valid credentials result in navigation to the dashboard and display user profile details.

2. Negative Authentication Scenarios
Verify that the application properly displays user-friendly, secure error messages:
- Invalid password: `Invalid email or password` (never specify which one failed, preventing user enumeration!).
- Nonexistent email.
- Empty fields (client-side form validation).

3. Logout Workflow
Verify that clicking Logout:
1. Invalidates the session on the server.
2. Clears browser cookies and localStorage.
3. Redirects to the public login or landing page.
4. Prevents navigating back via the browser Back button!

4. Protected Route Guarding

Attempting to navigate directly to `/app` or `/settings` while unauthenticated must redirect to `/login`.

Testing Protected Route Redirection (Playwright test verifying that guest users are blocked from dashboard.)

```
test('unauthenticated visitor is redirected from dashboard to login', async ({ page }) => {
  await page.goto('/app');
  await expect(page).toHaveURL(/.*login/);
  await expect(page.getByRole('heading', { name: /sign in|log in/i })).toBeVisible();
});
```

In an enterprise test suite with 200 tests, logging in via the UI before every single test wastes 15–20 minutes of CI time!

What Is `storageState`?

Playwright can snapshot the authenticated state (cookies and localStorage) of a browser context and save it to a local JSON file:

```javascript

// auth.setup.js

const { test: setup, expect } = require('@playwright/test');

```
setup('authenticate as test student', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill(process.env.TEST_STUDENT_EMAIL);
  await page.getByLabel('Password').fill(process.env.TEST_STUDENT_PASSWORD);
  await page.getByRole('button', { name: 'Sign In' }).click();

  await expect(page.getByRole('heading', { name: /dashboard|my learning/i })).toBeVisible();

  // Save session cookies & localStorage to disk!
  await page.context().storageState({ path: 'playwright/.auth/student.json' });
});
```

Reusing `storageState` in `playwright.config.js`

```javascript

// playwright.config.js

module.exports = {

```
  projects: [
    // 1. Setup project runs first to authenticate once
    { name: 'setup', testMatch: /.*\.setup\.js/ },
    // 2. Main tests reuse the saved authentication state
    {
      name: 'chromium',
      dependencies: ['setup'],
      use: {
        storageState: 'playwright/.auth/student.json',
      },
    },
  ],
};
```

Avoiding Shared Mutable State

If a test modifies the user account (e.g. changes the password, updates the profile name, or deletes account data), do NOT use a shared `storageState`!

Use dedicated test users or create fresh isolated accounts for destructive tests.

Direct Test with Injected Storage State (A test that opens directly to an authenticated dashboard.)

```
test('authenticated user sees enrolled courses', async ({ page }) => {
  // Page opens pre-authenticated! Zero login steps required!
  await page.goto('/app');
  await expect(page.getByText('My Enrolled Courses')).toBeVisible();
});
```

Security and test isolation are non-negotiable standards in automated testing.

1. Zero Plaintext Passwords in Code
Never commit passwords, API keys, or access tokens to GitHub or GitLab:
```javascript
// 'L DANGEROUS SECURITY VULNERABILITY:
await page.getByLabel('Password').fill('SuperSecretP@ssword2026!');
// ' SECURE BEST PRACTICE:
await page.getByLabel('Password').fill(process.env.TEST_USER_PASSWORD || '');
```

**2. Managing `.env` and `.gitignore`**
**Always maintain a `.env.example` for documentation, but keep `.env` and `playwright/.auth/` in `.gitignore`:**
**```text**
**# .gitignore**
**.env**
**.env.local**
**playwright/.auth/**
**test-results/**
**playwright-report/**
```

3. Test Isolation Principles
1. **No Test Order Dependence: Test B must pass whether it runs before Test A, after Test A, or simultaneously on a different worker.**
2. **Dedicated Test Accounts: Never use real customer accounts or shared production credentials.**
3. **Data Teardown / Sandbox Data: Ensure tests clean up records or operate in disposable sandbox tenants.**

Safe Credential Loading Pattern (Loading credentials with verification.)

```
const email = process.env.TEST_USER_EMAIL;
const password = process.env.TEST_USER_PASSWORD;

if (!email || !password) {
  throw new Error('Missing required test credentials: TEST_USER_EMAIL and TEST_USER_PASSWORD must be defined.');
}
```

requirements.

Assignment Starter Code (Day 11)

```
// tests/authentication_and_session_suite.spec.js
const { test, expect } = require('@playwright/test');

// Dedicated, safe fictional credentials for automated testing
const TEST_CREDENTIALS = {
  validUser: {
    email: process.env.TEST_STUDENT_EMAIL || 'student.automation@rootaccess.org',
    password: process.env.TEST_STUDENT_PASSWORD || 'RootAccess@2026',
    fullName: 'Automated Student',
  },
  invalidUser: {
    email: 'unregistered.user@rootaccess.org',
    password: 'WrongPassword123!',
  },
};

test.describe('Authentication, Session Lifecycle & Route Guards', () => {

  // 1. Unauthenticated Route Guard Testing
  // Explicitly ensure this test starts with zero stored cookies or tokens!
  test.describe('Unauthenticated Guest Tests', () => {
```

Q1. How does Playwright's storageState mechanism work, and what are its advantages over logging in before every test?

[Difficulty: Intermediate] Evaluation Focus: Authentication architecture and execution performance.

Model Answer: Playwright's `storageState` captures the authenticated state—specifically HTTP cookies, origin local storage, and session storage—from a browser context and persists it to a JSON file. Using Playwright's setup project feature, we perform the UI login flow once before the suite runs, save the session to `playwright/.auth/user.json`, and configure subsequent projects to launch with this storageState pre-loaded. This eliminates hundreds of redundant UI logins, reduces test suite duration by up to 80%, and prevents rate-limiting on login endpoints.

Follow-Up Question: "When would you deliberately NOT use storageState?"

Q2. How do you test that an unauthenticated user cannot access protected application routes?

[Difficulty: Beginner] Evaluation Focus: Route guarding, authorization testing, and web-first assertions.

Model Answer: In an isolated, unauthenticated test (a browser context with empty storageState), I call `await page.goto("/protected-dashboard")`. I assert that the application intercepts the request and redirects the user to the login route using `await expect(page).toHaveURL(/.*login/)`. I also verify that the login form heading is visible and that no sensitive dashboard data or account details are rendered in the DOM.

Follow-Up Question: "How do you verify role-based access control (RBAC) where a Student tries to visit /admin?"

Q3. How do you manage passwords and API secrets safely in an enterprise Playwright CI/CD pipeline?

[Difficulty: Intermediate] Evaluation Focus: Security posture, environment variables, and credential hygiene.

Model Answer: Credentials must never be committed to source code or plain test data files. In CI/CD (such as GitHub Actions), credentials are stored as encrypted repository secrets and injected into the test container as environment variables. In test scripts, we access them via `process.env.TEST_PASSWORD`. Additionally, we maintain `.env` and `playwright/.auth/` entries in `.gitignore` to prevent accidental credential leakage in pull requests.

Follow-Up Question: "What precautions should you take regarding Playwright Trace Viewer and screenshots when dealing with credentials?"

Q4. What is test isolation, and why is shared mutable state dangerous in parallel Playwright execution?

[Difficulty: Intermediate] Evaluation Focus: Test independence, concurrency, and parallelism.

Model Answer: Test isolation means that each test runs independently without relying on the state created by other tests. When Playwright executes tests in parallel across multiple worker processes, tests run concurrently in non-deterministic order. If tests share the same mutable account (e.g. Test A updates the profile name or password while Test B asserts the original profile), race conditions occur and tests fail intermittently. We prevent this by ensuring tests either use read-only shared states or generate isolated, dedicated user accounts.

Follow-Up Question: "How does Playwright BrowserContext provide built-in isolation between tests?"

Q5. How would you test session expiration or logout handling in Playwright?

[Difficulty: Scenario-Based] Evaluation Focus: Cookie manipulation, API session invalidation, and logout flow.

Model Answer: For logout, I click the user avatar, click "Sign Out", and assert that the user is redirected to `/login`. To verify true session destruction, I attempt to navigate back to `/app` using `page.goto()` and assert that the application denies entry and re-redirects to `/login`. For session expiration, I either programmatically clear the authentication cookie using `await page.context().clearCookies()` or use Playwright's `request` API to invalidate the session token on the backend, then perform a page action to assert that an "Expired session - please log in" modal or redirect is triggered.

Follow-Up Question: "How does page.context().clearCookies() differ from clearing browser cache?"

1. What does Playwright storageState store when saved from a browser context?

Key Concept: storageState extracts all active cookies and origin-specific localStorage key-value pairs, enabling subsequent contexts to instantiate in an already-logged-in state.

2. Where should credentials like TEST_USER_PASSWORD be accessed in a production Playwright test?

Key Concept: Using process.env keeps sensitive credentials out of version control and allows secure injection via CI/CD secret managers.

3. Which file must include playwright/.auth/ to prevent committing cached session tokens?

Key Concept: .gitignore instructs Git to ignore the playwright/.auth/ directory so active authentication tokens are never pushed to remote repositories.

4. What is the purpose of an authentication "setup project" in playwright.config.js?

Key Concept: A setup project runs before other projects, logs in, and produces a storageState file that subsequent test projects declare as a dependency and reuse.

5. How do you verify that an unauthenticated user is blocked from viewing a dashboard?

Key Concept: Testing route protection requires an unauthenticated context attempting to visit a protected URL and verifying that route guards redirect to the login page.

6. Why is using a shared authenticated user dangerous for tests that change passwords or update user profile data?

Key Concept: Shared mutable state causes race conditions when tests run in parallel. Destructive or state-mutating tests must use dedicated, isolated user accounts.

7. Which method clears all cookies from the current Playwright BrowserContext?

Key Concept: context.clearCookies() clears all stored cookies from the browser context, which is useful for testing immediate session invalidation or logout behavior.

8. What is the difference between Authentication (AuthN) and Authorization (AuthZ)?

Key Concept: AuthN establishes user identity (login), while AuthZ enforces security policies and role permissions on protected resources.

9. How do you configure a specific test to bypass the global storageState and run as an unauthenticated guest?

Key Concept: test.use({ storageState: { cookies: [], origins: [] } }) overrides project-level auth configuration, providing a clean unauthenticated context for guest tests.

10. What is the most secure way to handle test user credentials for local test runs?

Key Concept: Storing local secrets in a gitignored .env file and loading them into process.env keeps secrets secure on developer machines without risking repo leaks.